



Satellite Land Cover Classifier

System Analysis & Design

Problem & Objectives • Use Cases • Requirements • Architecture • Data Modeling • System Behavior • UI/UX • Deployment



System Analysis & Design

Problem & Objectives • Use Cases • Requirements • Architecture

Problem Statement & Objectives

Why this system exists, and what it is designed to achieve



The Problem

Manually classifying land cover from satellite imagery is slow, expensive, and requires specialized remote-sensing expertise. Organizations need a fast, automated way to turn raw Sentinel-2 imagery into usable land-cover labels — without every user needing to be a machine-learning practitioner.



Success Looks Like

A user uploads a valid 14-band image, picks a model, and within seconds sees a labeled class, a confidence score, and the full probability breakdown — with a clear error if the file isn't genuine multispectral data.

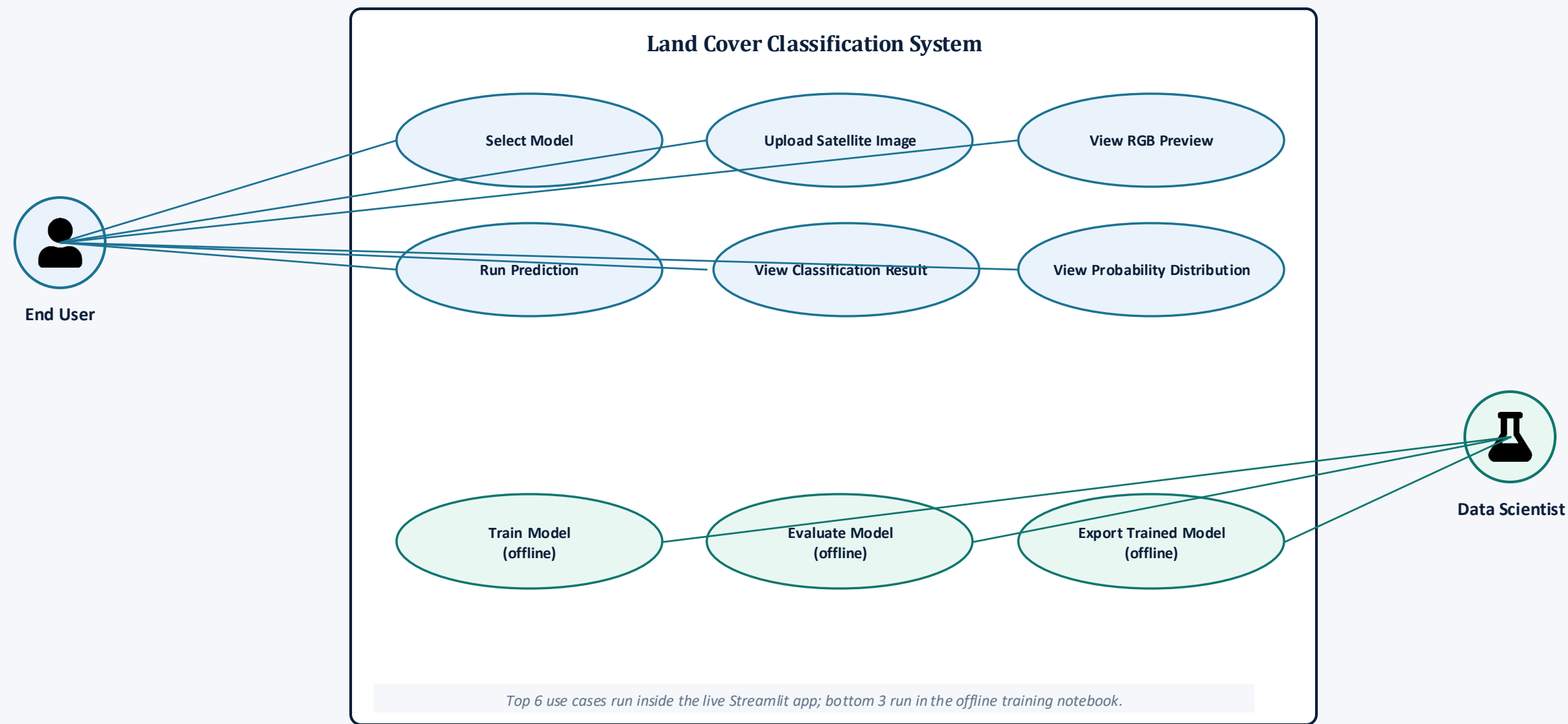


Project Objectives

- Train and fairly compare four CNN architectures — SpectrumNet, SimpleCNN, ResNet50, EfficientNet — on 14-band EuroSAT imagery across 10 land-cover classes.
- Deliver a self-serve web app so a non-ML user can classify a Sentinel-2 image in a few clicks.
- Let users choose between transfer-learning models (higher accuracy) and lightweight custom models (faster, smaller).
- Keep predictions trustworthy by accepting only genuine 13-band multispectral .tif imagery.
- Keep the model roster easy to extend as new architectures are trained.

Use Case Diagram

Actors and their interactions with the system



Use Case Descriptions

Key interactions in detail

ID	Use Case	Actor	Description
UC-1	Select Model	End User	User picks one of four registered models (transfer-learning or lightweight) before uploading an image.
UC-2	Upload Satellite Image	End User	User uploads a .tif/.tiff file; system validates it has exactly 14 bands before proceeding.
UC-3	Run Prediction	End User	User triggers inference; the selected model (loaded from cache or disk) classifies the preprocessed image.
UC-4	View Classification Result	End User	System displays the predicted class, its description, and a confidence percentage.
UC-5	Train Model	Data Scientist	Offline: train one of four CNN architectures on the EuroSAT dataset split (80/10/10).
UC-6	Evaluate Model	Data Scientist	Offline: score a trained model's Accuracy, Precision, Recall and F1 on the held-out test set.

Functional Requirements

What the system must do



Training Pipeline

- Load & preprocess 14-band EuroSAT imagery
- Stratified 80/10/10 train/val/test split
- Train 4 architectures on a (64,64,13) input
- Score Accuracy, Precision, Recall & F1
- Persist each model as a .h5 file



Image Input

- Accept only .tif/.tiff uploads
- Validate exactly 14 bands present
- Reject invalid files with a clear error
- Resize to 64×64 for inference
- Render a true-color RGB preview (bands 4/3/2)



Prediction & Results

- Let the user pick any of 4 registered models
- Cache loaded models for fast reuse
- Show predicted class, description & confidence
- Show full probability distribution (all 10 classes)
- Handle model-load errors gracefully

Non-Functional Requirements

Constraints the system must satisfy



Performance

Inference returns within a few seconds after model load; caching keeps model-switching fast.



Reliability

Invalid uploads (wrong format, wrong band count) are rejected with a readable error, never a crash.



Maintainability

Model registry keeps file paths, icons and metadata centralized, not scattered in UI code.



Usability

Model selection, upload, preview and results stay in clearly separated, uncluttered sections.



Reproducibility

Training uses a fixed random seed (42) so data splits and results are reproducible.

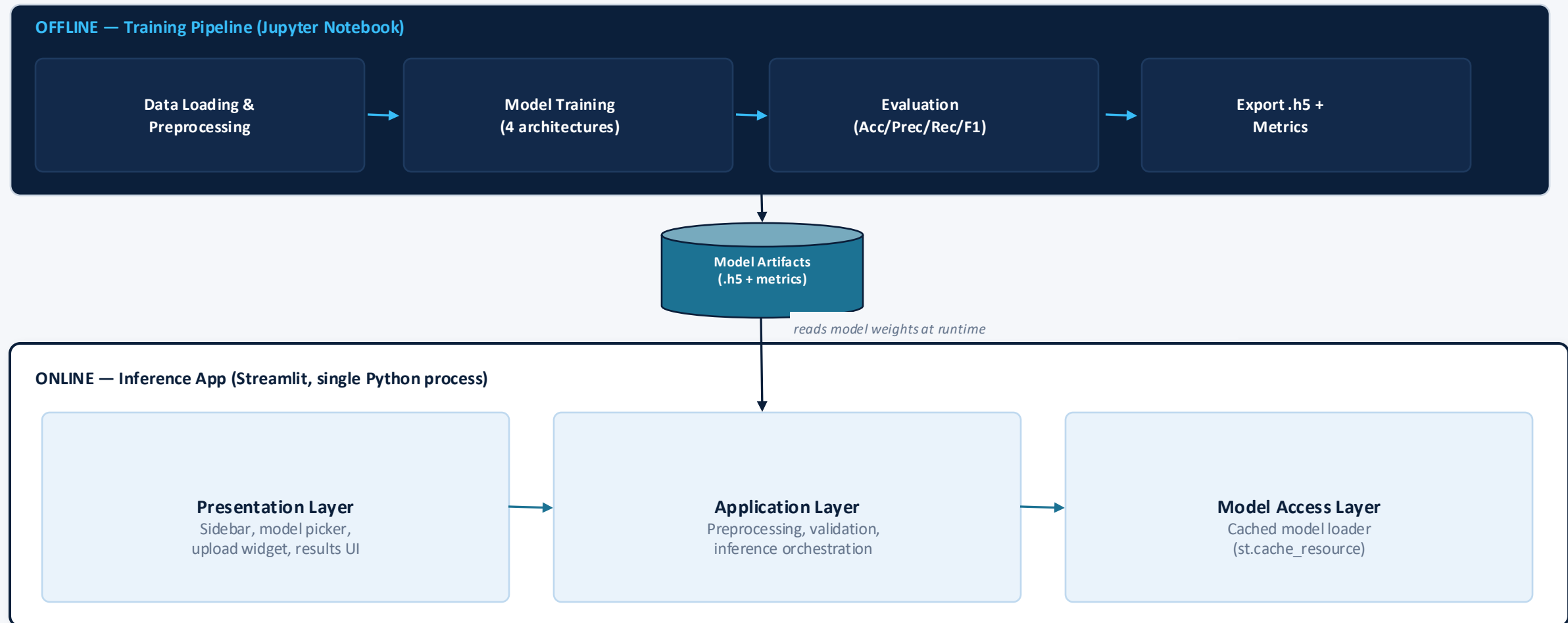


Portability

Model files load relative to the app's working directory — no hard-coded absolute paths.

Software Architecture

Layered monolith — offline training pipeline feeding an online inference app



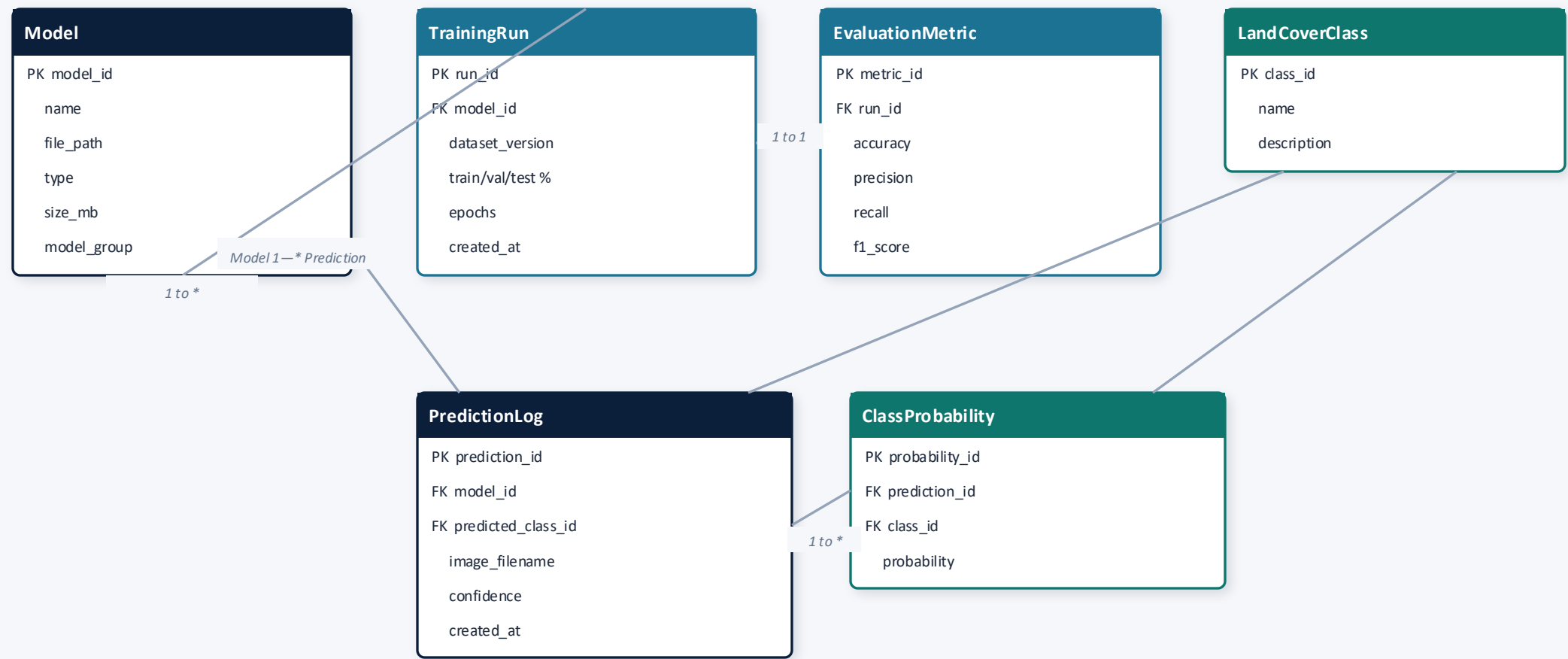
2

Database Design & Data Modeling

ER Diagram • Logical & Physical Schema

ER Diagram

Proposed schema for experiment tracking & prediction logging (the app is currently stateless — this formalizes the data it already produces)



Logical & Physical Schema

Tables, keys, and normalization notes (proposed 3NF schema)

Table	Primary Key	Key Columns	Normalization Notes
Model	model_id (PK, INT)	name VARCHAR(50), file_path VARCHAR(255), type VARCHAR(30), size_mb FLOAT, model_group VARCHAR(20)	One row per registered architecture (4 rows today)
TrainingRun	run_id (PK, INT)	model_id (FK), dataset_version VARCHAR(20), split ratios, epochs INT, created_at DATETIME	Separated from Model to allow re-training history (3NF)
EvaluationMetric	metric_id (PK, INT)	run_id (FK), accuracy/precision/recall/f1 FLOAT	1:1 with TrainingRun — split out to avoid repeating metric columns on every run row
LandCoverClass	class_id (PK, INT)	name VARCHAR(30), description VARCHAR(120)	Static reference table — the 10 EuroSAT classes
PredictionLog	prediction_id (PK, INT)	model_id (FK), predicted_class_id (FK), image_filename, confidence FLOAT, created_at DATETIME	One row per inference request made through the app
ClassProbability	probability_id (PK, INT)	prediction_id (FK), class_id (FK), probability FLOAT	Normalized out of PredictionLog to avoid 10 repeating probability columns (removes repeating groups → 1NF/2NF)

3

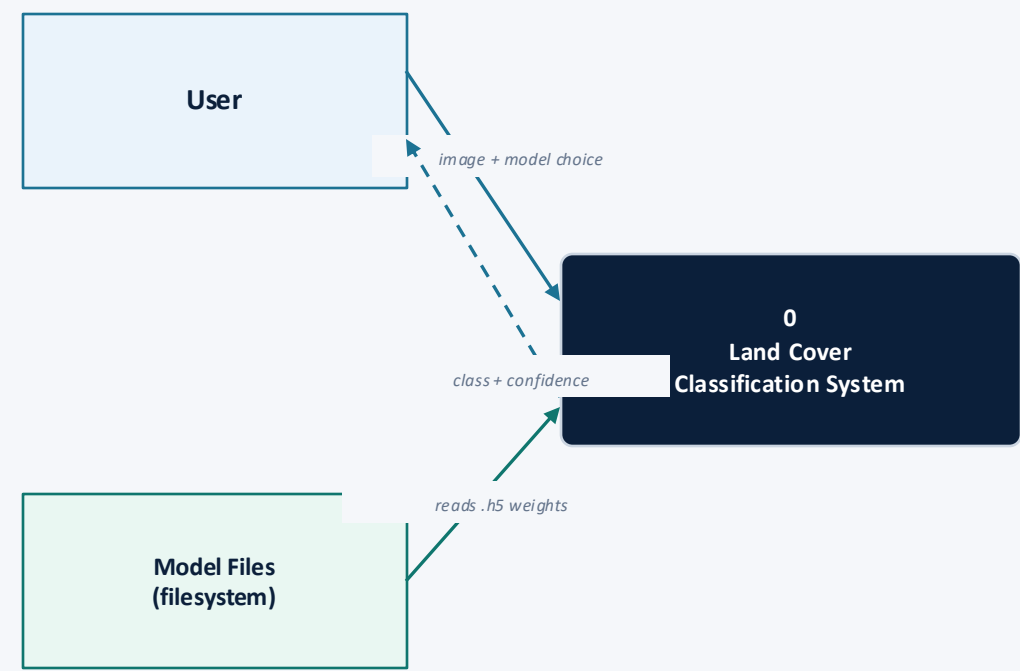
Data Flow & System Behavior

DFD • Sequence • Activity • State • Class Diagram

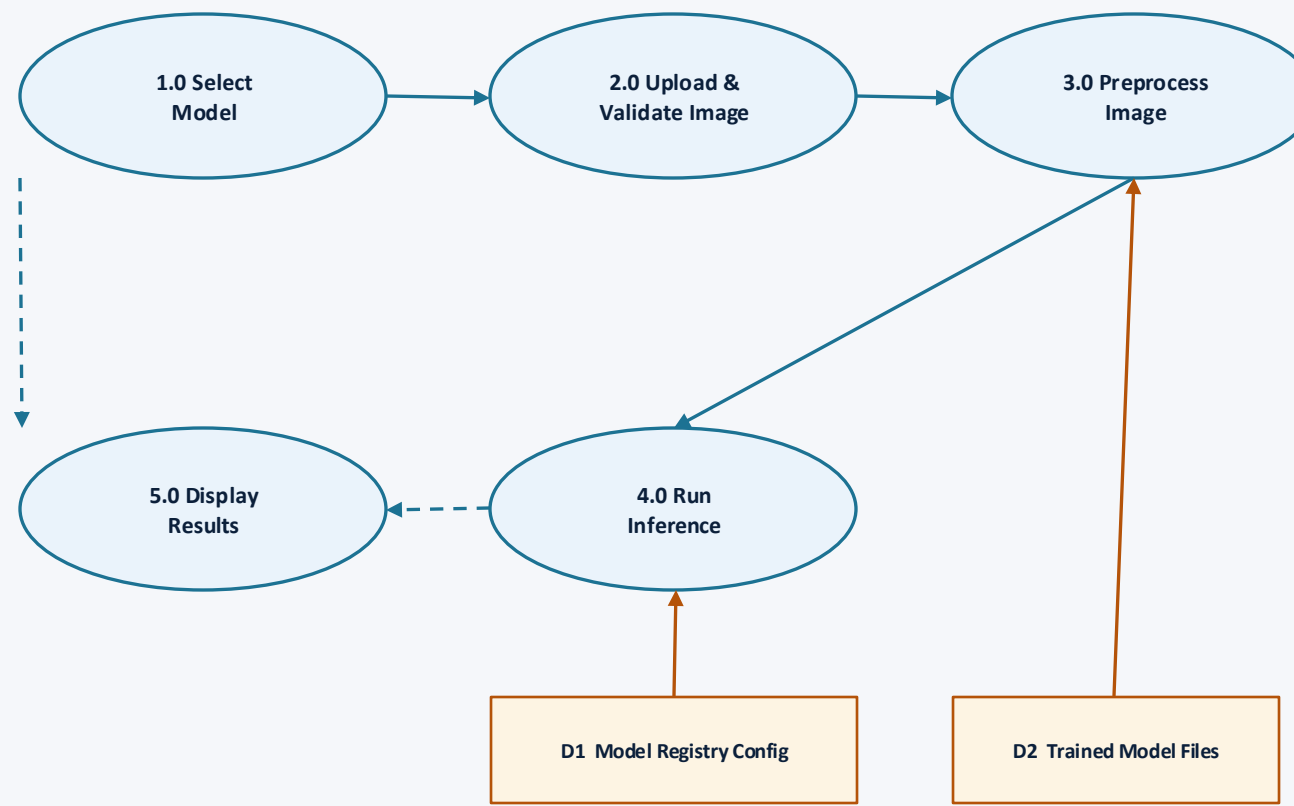
Data Flow Diagram

Context level (0) and Level-1 process breakdown

CONTEXT DIAGRAM (LEVEL 0)

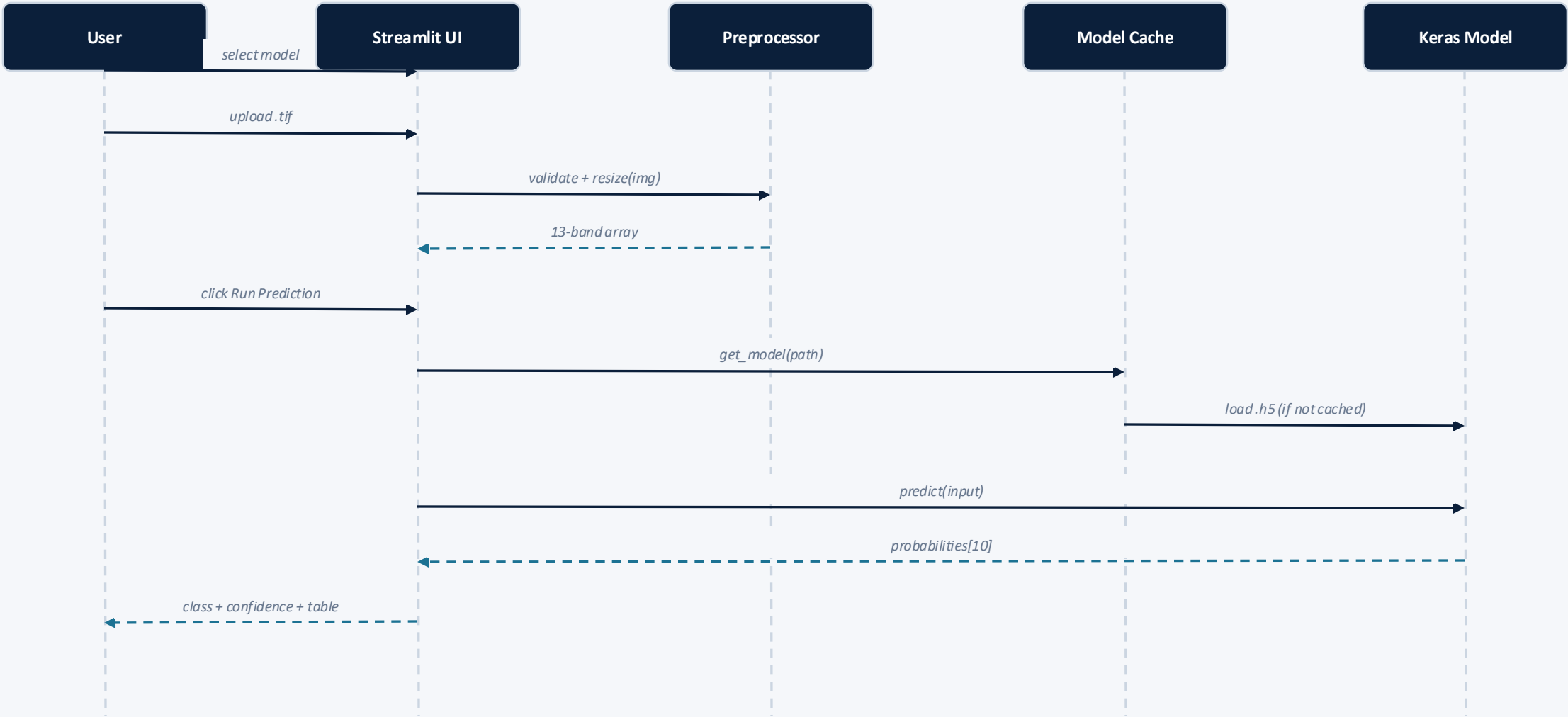


LEVEL 1 — PROCESS BREAKDOWN



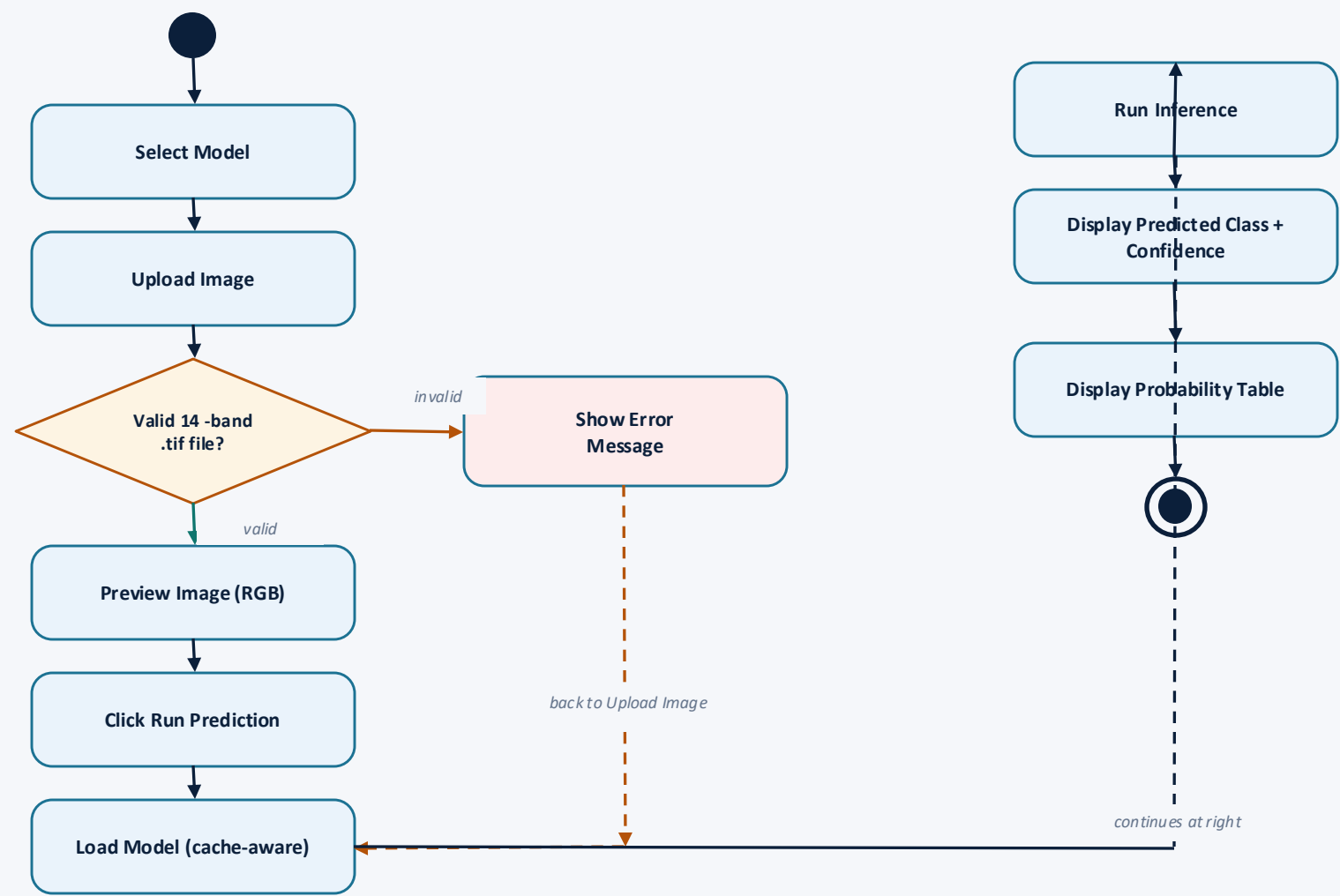
Sequence Diagram

Run Prediction — from click to result



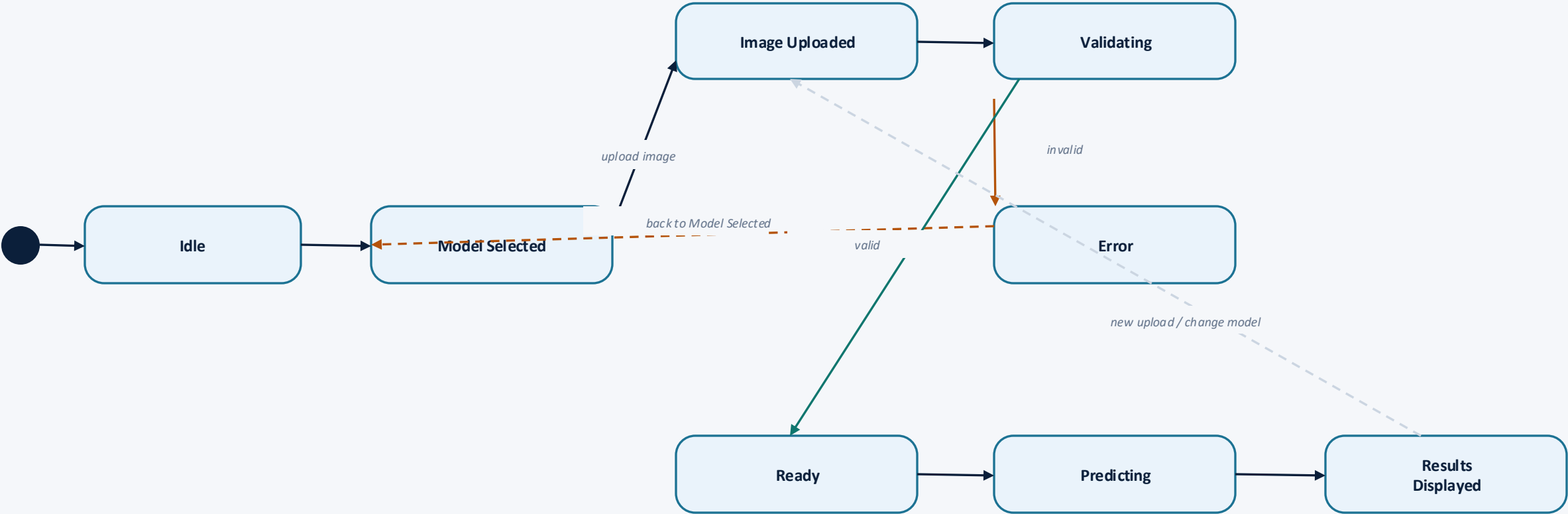
Activity Diagram

End-user workflow, from opening the app to seeing a result



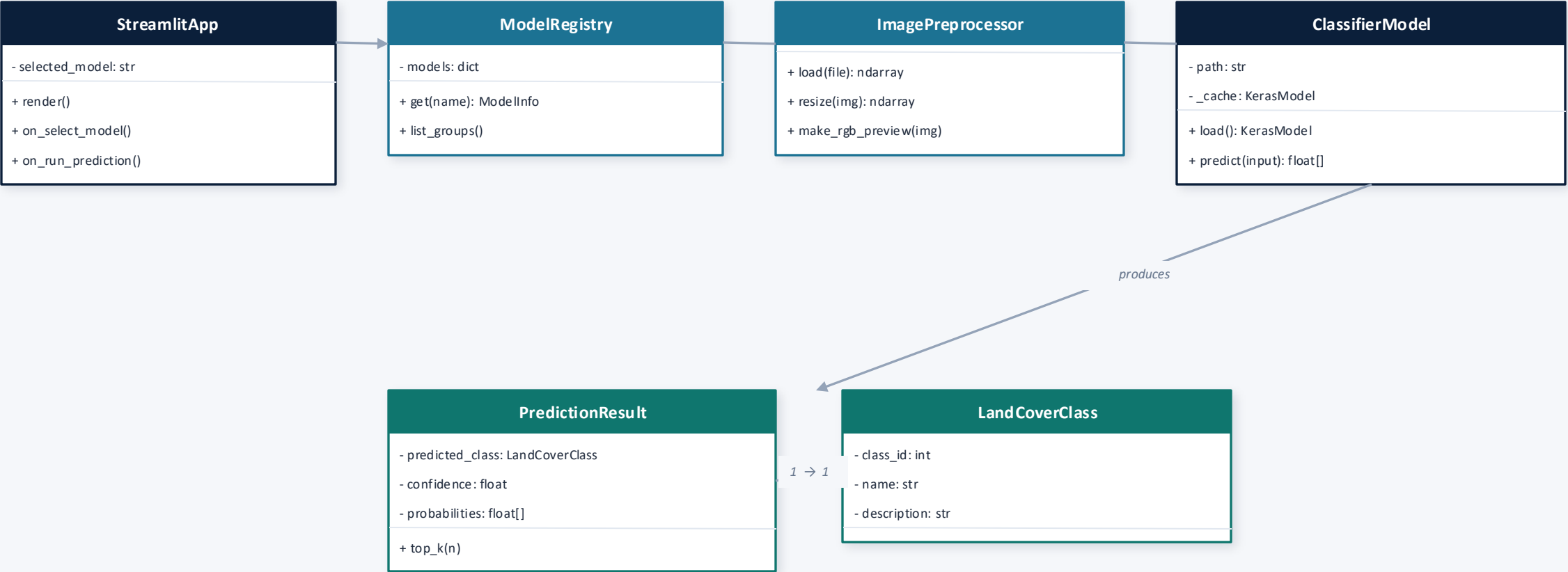
State Diagram

Lifecycle of a prediction session in the app



Class Diagram

Core classes behind the Streamlit app



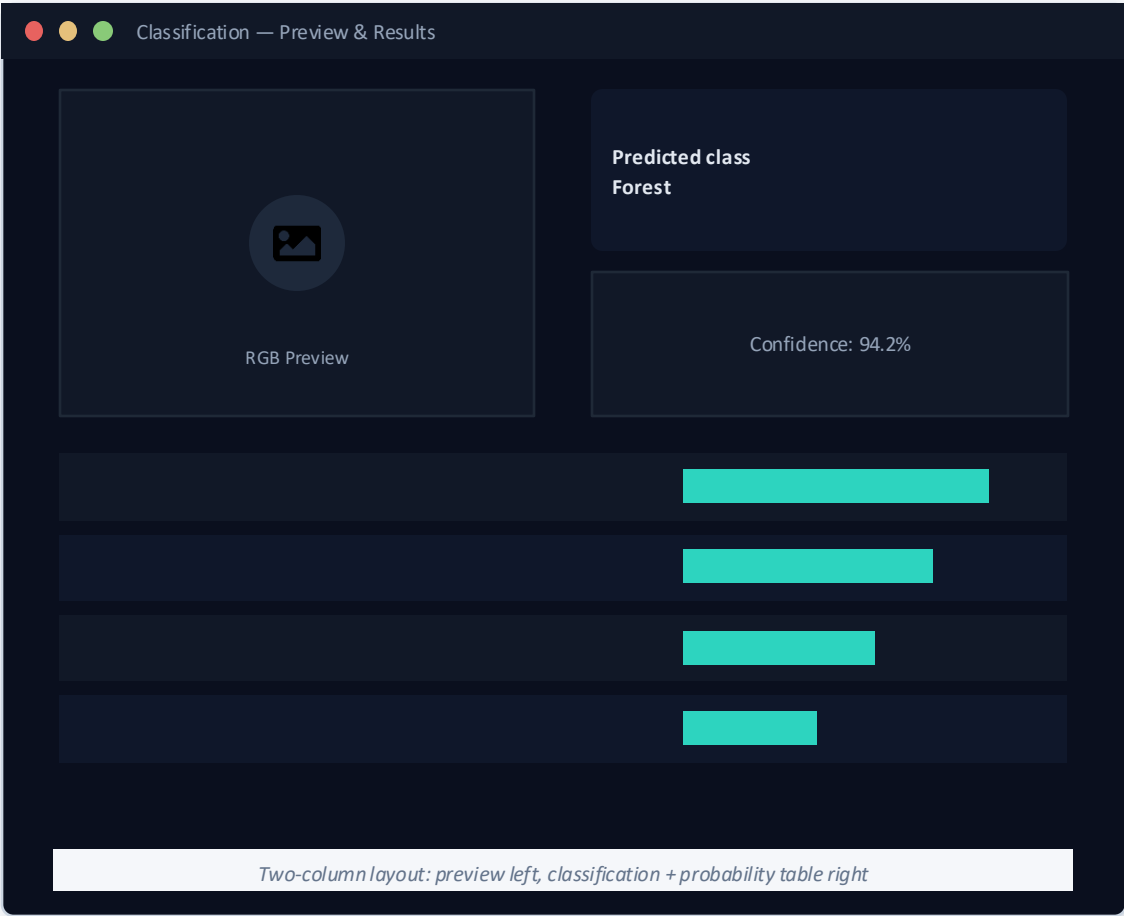
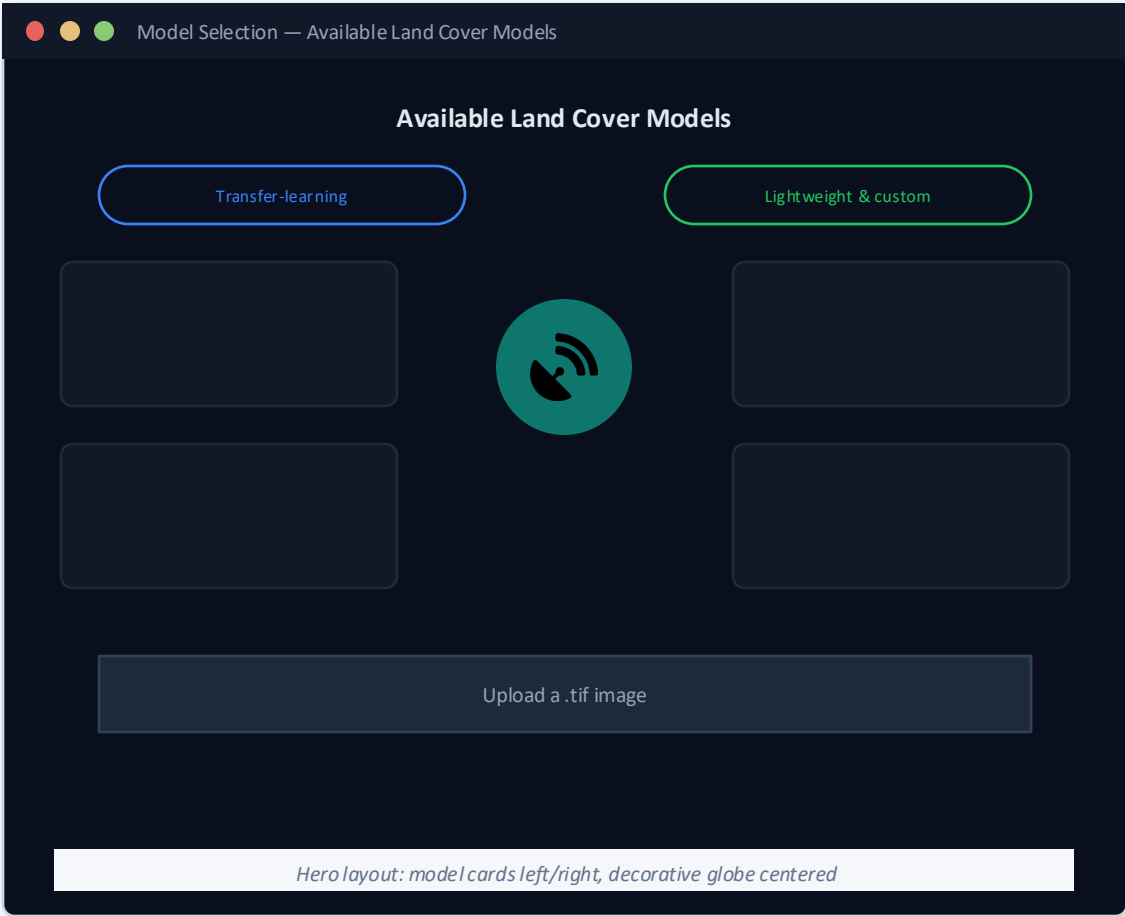
4

UI/UX Design & Prototyping

Wireframes & Mockups • UI/UX Guidelines

Wireframes & Mockups

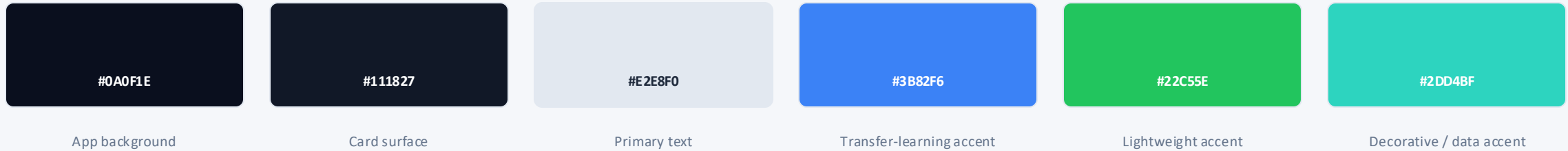
The app's two primary screens



UI/UX Guidelines

Design principles behind the interface

COLOR PALETTE



Typography

- System sans-serif stack, bold (700) headings with tight letter-spacing
- Card titles ~1rem bold; supporting detail text ~0.85rem, muted gray
- Monospace used only for file paths and shapes (e.g. model file names)



Accessibility & Feedback

- Selected model card uses both a border-color change AND a checkmark label — never color alone
- Errors use icon + red/amber tone + explicit text, not color alone
- Dark theme maintains high text/background contrast (#E2E8F0 on #0A0F1E)
- Disabled state on the already-selected model's button prevents redundant clicks

5

System Deployment & Integration

Technology Stack • Deployment Diagram • Component Diagram

Technology Stack

What the system is built with



Frontend / UI

- Streamlit (Python web framework)
- Custom CSS (dark theme, cards)
- Inline SVG (decorative globe graphic)



ML & Data Handling

- TensorFlow / Keras (models)
- NumPy, Pandas (arrays, tables)
- scikit-image (resize)
- tiff file (14-band .tif I/O)

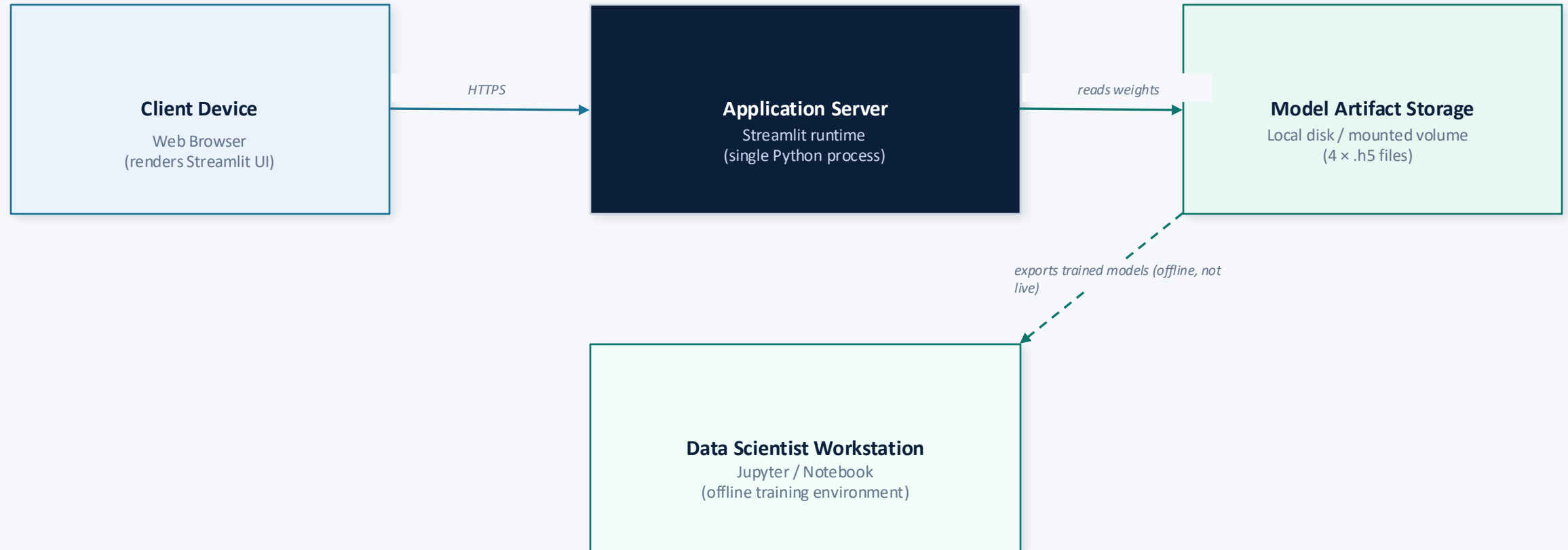


Training Environment

- Jupyter Notebook
- scikit-learn (split, metrics, label encoding)
- HDF5 (.h5) model artifact format
- Python 3 runtime throughout

Deployment Diagram

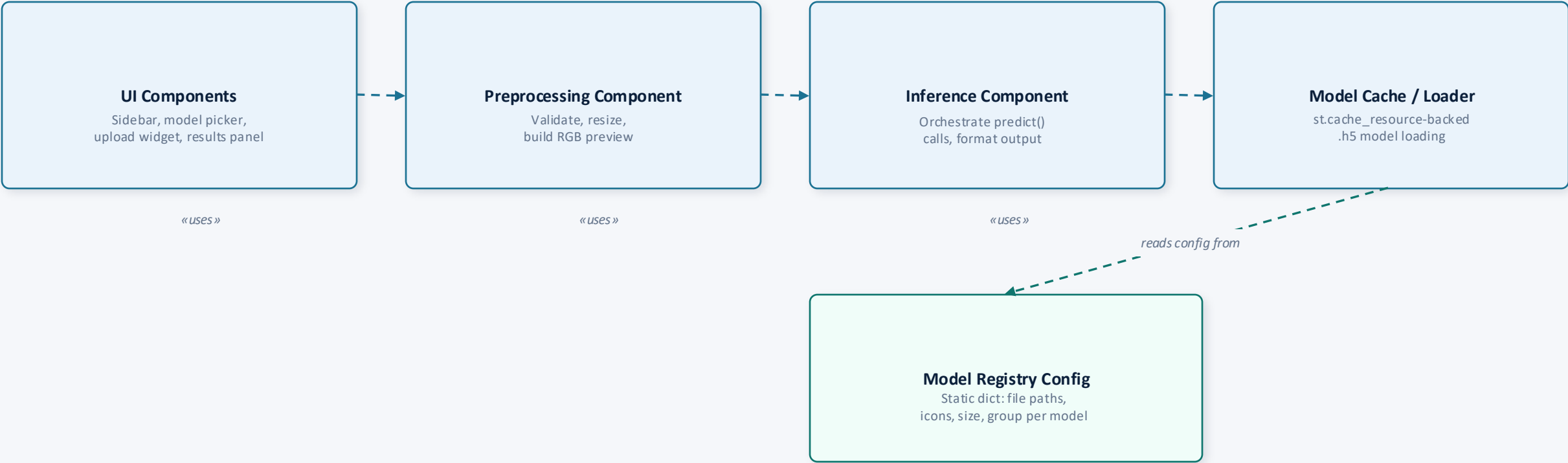
How components map onto physical / runtime nodes



No REST API or database tier exists yet — the browser talks directly to the single Streamlit process, which reads model files from local storage.

Component Diagram

High-level components and their dependencies



6

Additional Deliverables

API Documentation • Testing & Validation • Deployment Strategy

API Documentation

No REST API exists today — this documents the internal function contract instead



The app is a single Streamlit process with no HTTP/REST layer — these are the internal Python functions that a future API would wrap.

Function	Input	Output	Purpose
<code>preprocess_image(file)</code>	Uploaded .tif file	(<code>raw_img</code> : ndarray, <code>model_input</code> : ndarray)	Reads, validates 14 bands, resizes to 64×64
<code>make_rgb_preview(img)</code>	14-band ndarray	RGB ndarray (0–1 range)	Builds a true-color preview from bands 4/3/2
<code>get_model(path)</code>	Model file path (str)	Loaded Keras model (cached)	Loads a .h5 model, cached via <code>st.cache_resource</code>
<code>model.predict(input)</code>	Preprocessed 4D array	float[10] class probabilities	Runs inference for the selected model

Future extensibility: a `POST /predict` endpoint could wrap `preprocess_image()` + `model.predict()` directly, returning the same JSON shape shown in the results panel.

Testing & Validation

How correctness is verified at each level



Unit Tests

- Band-count validation rejects non-14-band arrays
- Resize output always (64,64,13)
- Model registry lookups return correct file paths
- RGB preview picks correct bands (4/3/2)



Integration Tests

- End-to-end: sample .tif → preprocess → predict → display
- Invalid file (wrong bands / format) surfaces a readable error, no crash
- Model switch triggers correct cached / fresh load



User Acceptance Testing

- Recruit representative analysts / end users
- Walk through model selection → upload → prediction
- Collect feedback on error-message clarity and confidence display
- Confirm results match expectations on known sample imagery

Deployment Strategy

Hosting, pipeline, and scaling considerations



Hosting Environment

- Streamlit Community Cloud, or a container (Docker) on internal infrastructure
- Model .h5 files bundled into the image or mounted as a volume alongside app.py
- Single-process deployment today — no load balancer or multi-instance setup yet



Scaling Considerations

- ResNet50 (~277 MB) is the heaviest artifact — lazy-load only the selected model to control memory
- Horizontal scaling behind a load balancer if concurrent users grow
- Model caching is per-process — a multi-instance deploy re-loads per instance

Deployment Pipeline

- 1 Push code to repository (app.py + requirements.txt)
- 2 CI installs dependencies and runs unit/integration tests
- 3 Build container image, bundling model .h5 artifacts
- 4 Deploy image to hosting environment
- 5 Smoke-test the live URL with a sample 14-band .tif

System Analysis & Design — Summary



Use Cases &
Requirements



Data Modeling
(ERD & Schema)



Data Flow &
Behavior (DFD, UML)



UI/UX Design



Deployment &
Integration



Testing &
Validation

Derived directly from the EuroSAT training notebook and the current Streamlit inference app (tif-only, 14-band validation).



Thank You